



Test-Driven Development for Code Generation

Mohammad Hajuj & Naief Hajuj

University of Waterloo, Canada - 11 jun 2024

מה הוא הרעיון המרכזי של TDD ?



מודלי שפה גדולים (LLMs) כמו GPT-4 ועוד יכולים לכתוב קוד , אבל האם הקוד נכון?

- ❑ הרעיון: במקום לבקש מהמודל לכתוב קוד ולקוות לטוב . **נספק לו טסטים מראש** ונבדוק אם הקוד עובר אותם.

וכאן עיקרון TDD — מועבר לעולם ה-AI.

מהו TDD ולמה זה חשוב?



בפיתוח רגיל, TDD מבטיח שהקוד עונה על הדרישות מהרגע הראשון. השאלה: האם אותו עיקרון עובד גם כשה-LLM כותב את הקוד?



האם טסטים מראש משפרים יצירת קוד ב LLMs?

שאלות המחקר המרכזיות במאמר

RQ1

עד כמה טוב יצירת קוד בסיסית?

RQ2

האם טסטים משפרים את התוצאה?

RQ3

האם כישלון בטסט מאפשר תיקון עצמי?

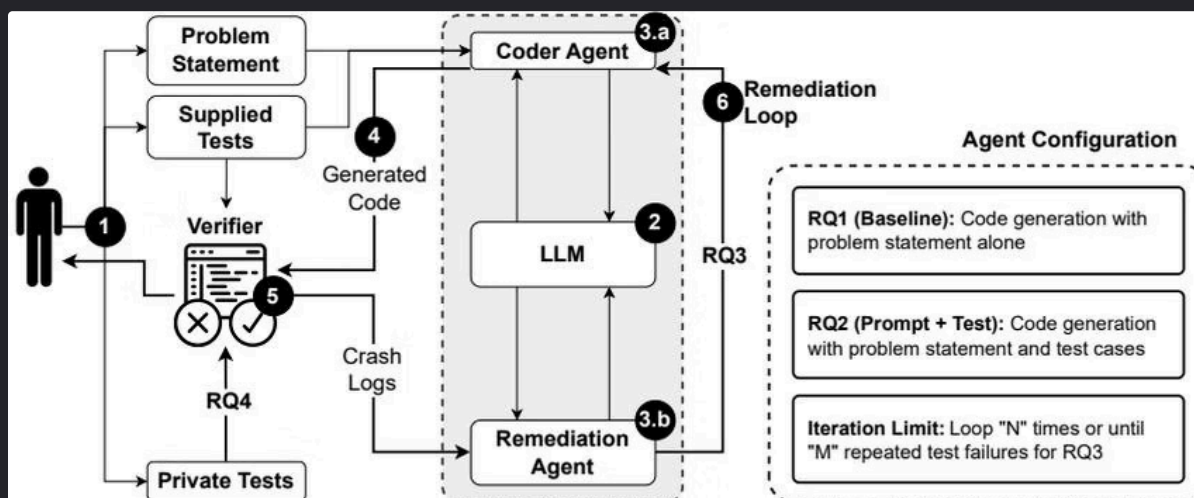
RQ4

האם הקוד עובד גם על טסטים לא ידועים?

איך גורמים ל LLM לכתוב קוד נכון יותר?

מסגרת TGen

זרימת העבודה



1. The developer provides a problem statement and test cases, just like in the TDD approach.
2. Codex Agent - The first agent generates the code by understanding the requirements, interpreting the tests, and producing a correct solution.
3. Verifier - The Verifier runs all tests on the generated code to check whether it actually works, not just looks correct.
4. Remediation Agent - If the code fails the tests, the Remediation Agent analyzes the failed tests and errors, then tries to fix the code through self-correction.
5. The loop continues until the code passes all tests or reaches 5 attempts.



חשוב לציין שהכלי לא מחליף את המפתח הוא מבטיח שהקוד שנוצר עובר את הטסטים שהמפתח כתב.

הנתונים ששומשו — על מה ניסו?



HumanEval

164 Python programming problems
With EvalPlus — 80x more tests for
each problem



MBPP

399 Python programming problems
With EvalPlus — 35x more tests for
each problem

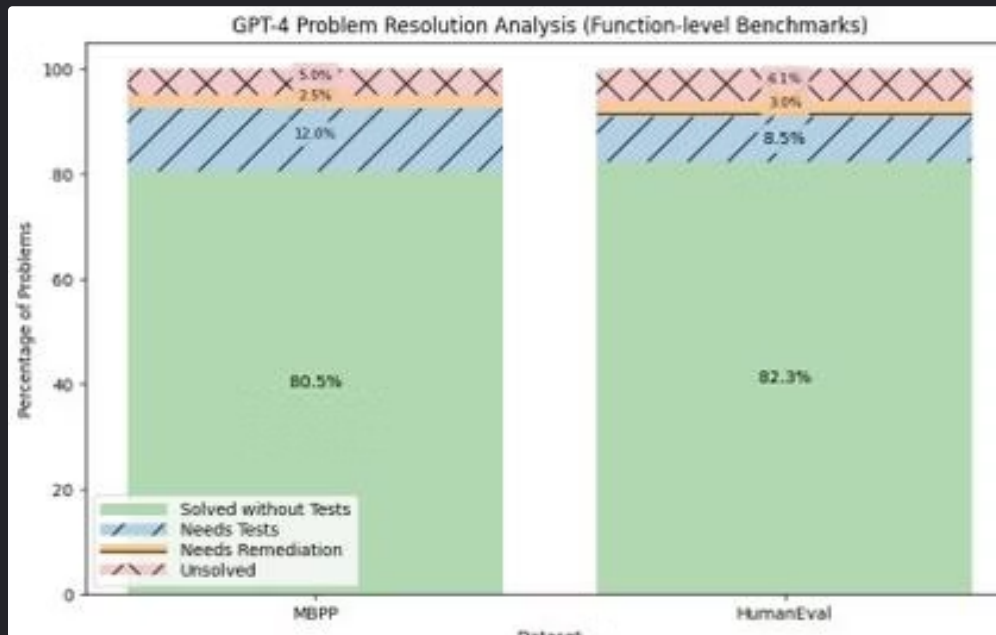


CodeChef (1100 בעיות)

1100 competitive programming
problems
Across 11 difficulty levels
Used to evaluate real-world
performance

איך טסטים מראש שיפרו את הקוד של GPT-4 - תוצאות הניסוי

ממצאים מרכזיים



בסיס (ללא טסטים)

MBPP: 80.5% · HumanEval: 82.3%

עם טסטים

HumanEval: +8.5%

MBPP: +12.8%

עם Remediation - שיפור נוסף :

HumanEval: +2.5%

MBPP: +3%

איך טסטים עוזרים ל LLM כדי לכתוב קוד נכון יותר?

$+=x$

נוסחאות מתמטיות

תיקון שגיאות בחישובים מונעים טעויות בחישובים ולוגיקה .



חתימת פונקציה

זיהוי אי-התאמה בין שם/פרמטרים לדרישה



מבני נתונים מורכבים

עוזרים להבין קלטים ופלטים מורכבים



Edge Cases (מקרי קצה)

טיפול במקרים שהמודל התעלם מהם בלי טסטים

Private Tests

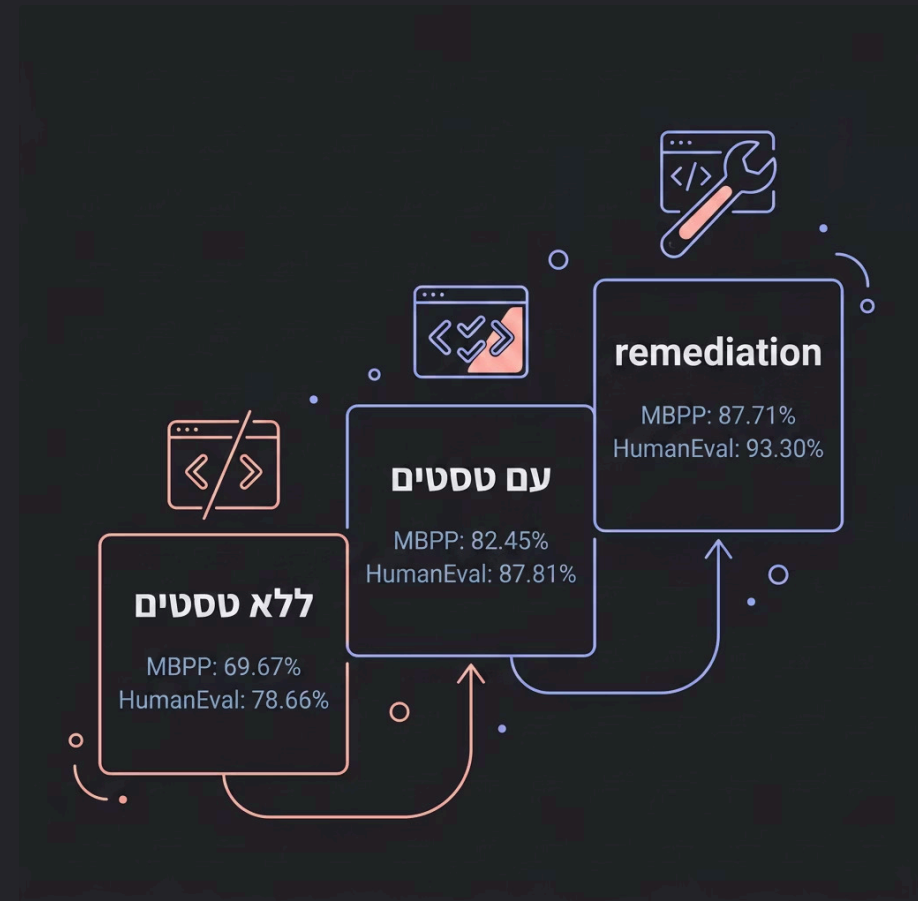
האם הקוד שנכתב באמת נכון ?

הבדיקה האמיתית

המערכת מוסיפה בדיקות נסתרות שהמודל אינו רואה בזמן יצירת הקוד, כדי לבדוק האם הקוד באמת נכון ולא רק מותאם לבדיקות הגלויות שניתנו לו .

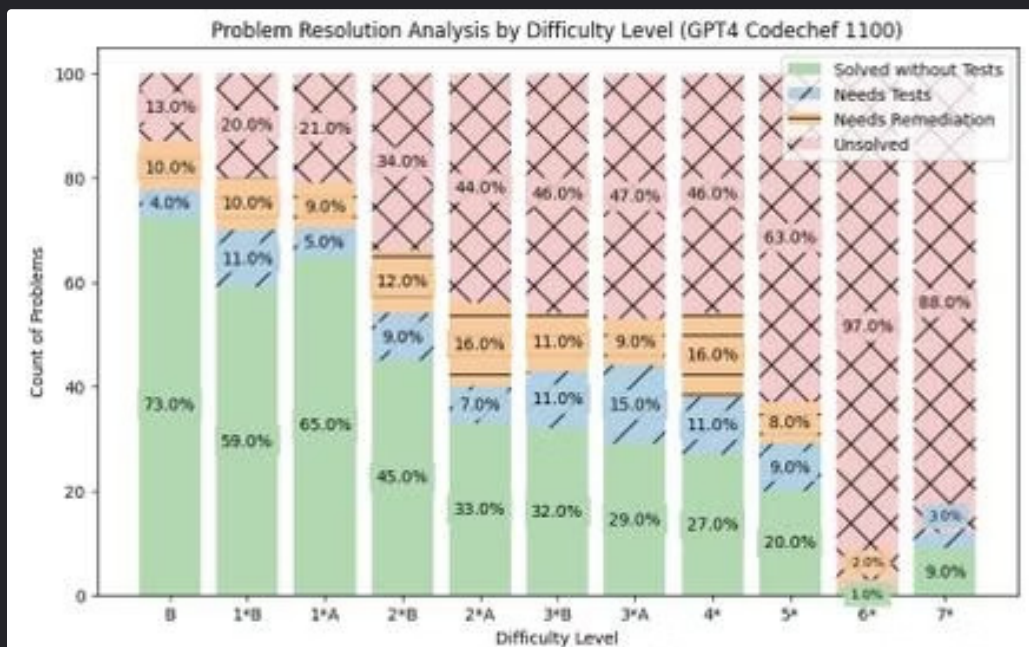
כן — גם מול בדיקות נסתרות, הוספת טסטים מראש שיפרה משמעותית את התוצאות: שיפור של 12.78% ב MBPP ושל 9.15% ב HumanEval.

המסקנה: קוד שלא ראה את כל הטסטים עדיין הצליח טוב יותר , כלומר הטסטים באמת שיפרו את ההבנה, לא רק את ההתאמה.



איך רמת הקושי של הבעיה משפיעה על הצלחת ה LLM ?

CodeChef – 1100 בעיות



- בעיות קלות: 73% נפתרות ללא טסטים
- בעיות קשות: רק 13% נפתרות ללא טסטים
- טסטים תרמו +7.72% בכל רמות הקושי
- הוסיף עוד +9.36% Remediation

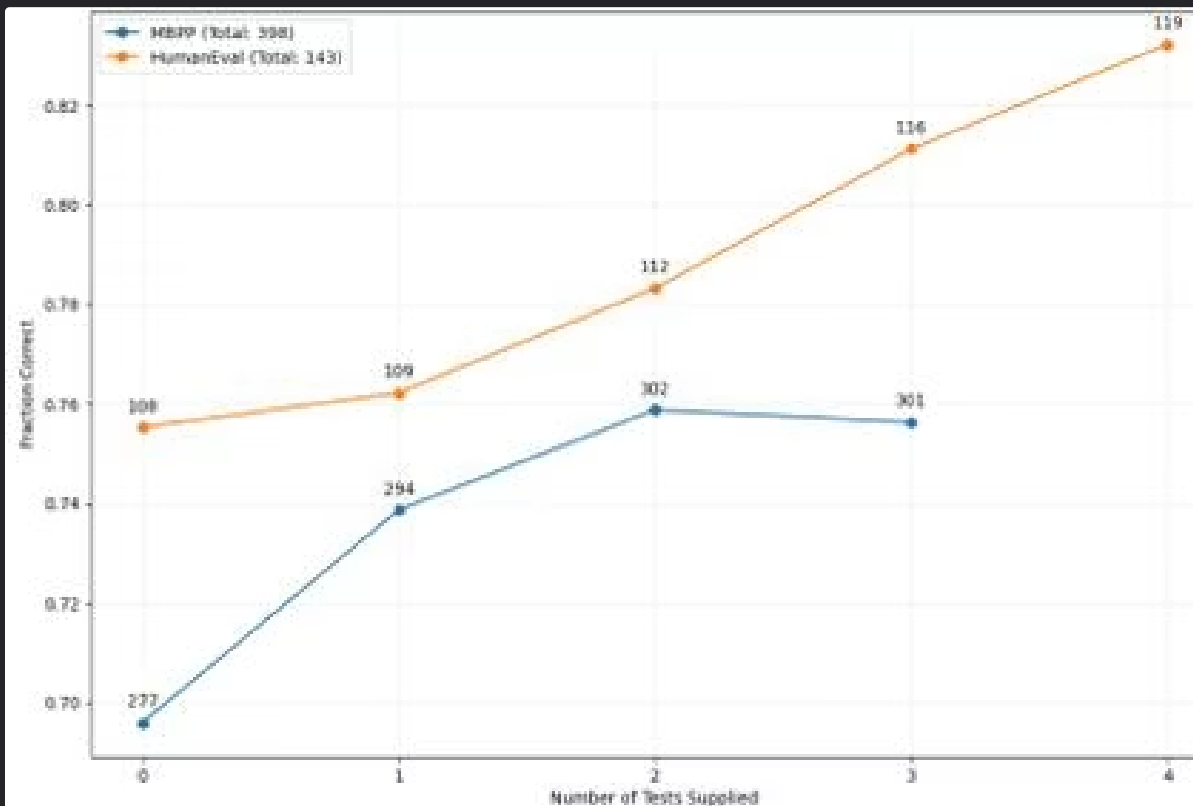
המסר המרכזי

ככל שהבעיה קשה יותר, טסטים הופכים לחשובים יותר אבל גם הם לא פותרים הכול !



47% מהבעיות הקשות נשארו ללא פתרון מה שמראה שגם במסגרת TDD יש גבול ליכולת ה LLM

כמה טסטים באמת מספיק ל LLM ?



ממצאים מעניינים

- יותר טסטים = תוצאות טובות יותר בדרך כלל
- ב MBPP כבר לא היה שיפור משמעותי אחרי 2-3 טסטים
- ב HumanEval — עלייה מתמשכת עד 4 טסטים. כלומר עוד טסטים עדיין עזרו.



Lost in the Middle

טסטים רבים עלולים לבלבל את המודל ולפגוע בתוצאה. כאשר יש יותר מדי מידע, המודל מתחיל להתעלם מחלק מהטקסט במיוחד מהאמצע ואז דווקא הביצועים נפגעים.

האם השיטה עובדת גם עם מודל פתוח כמו Llama 3 – ולא רק עם GPT-4?

תוצאות – Llama 3 70B

למרות שביצועי הבסיס של Llama 3 נמוכים יותר מ-GPT-4 (52.6% לעומת 80.5% ב-MBPP), הוספת טסטים ותהליך תיקון שיפרו אותו בצורה משמעותית יותר.

מסקנה: מודלים חלשים יותר מרוויחים אפילו יותר מגישת TDD.

עם Remediation

שיפור נוסף של +9.02%

עם טסטים - Llama 3 70B

+29.57% ב-MBPP

מסקנה: מודלים חלשים יותר מרוויחים אפילו יותר מגישת TDD

מה עדיין חסר ?

Benchmarks

הניסויים בוצעו על HumanEval ו MBPP ולא על פרויקטים אמיתיים.

איכות הטסטים קריטית

טסטים גרועים = קוד גרוע. המאמר לא מטפל בעניין של יצירת טסטים טובים.

יותר מדי טסטים פוגעים

תופעת "Lost in the Middle" — המודל מתחיל להתעלם מהקשר

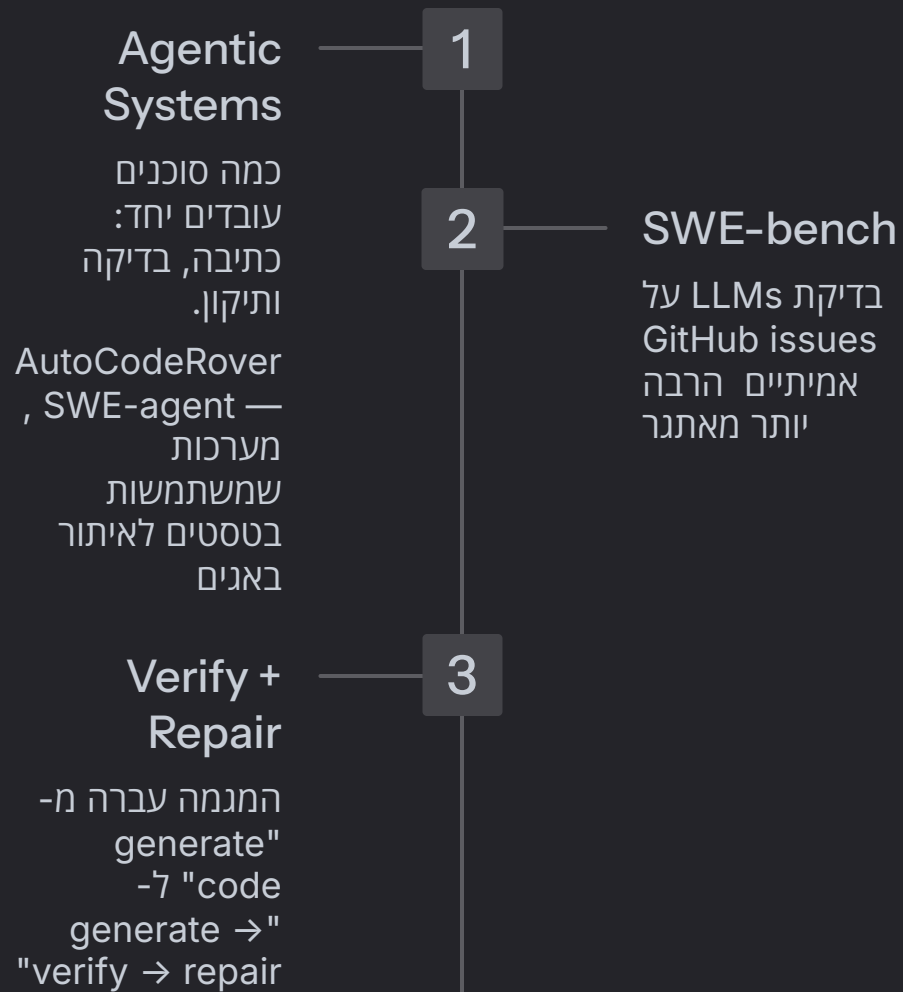
לא מתאים לכל בעיה

גם עם טסטים ותיקונים, בעיות מורכבות עם לוגיקה עמוקה נשארו ללא פתרון.



מה קרה מאז? – המגמה הגדולה

עולם פיתוח התוכנה עם AI התפתח יותר.



המאמר זיהה מגמה שהפכה לסטנדרט.

הוא הדגיש שהבסיס לגישה נכונה בעבודה עם LLMs הוא:
טסטים כמפרט, לא רק כבדיקה. היום מערכות רבות כבר עובדות בדיוק כך.

מסקנה: המאמר זיהה מגמה שהיום הפכה לסטנדרט:
טסטים כמפרט, לא רק כבדיקה.

לסיכום – TDD הופך את הקוד שהמודלים מייצרים לקוד אמין יותר.

המטרה אינה לגרום למודלים לכתוב קוד מהר יותר, אלא לגרום לו לכתוב קוד נכון ואמין יותר באמצעות טסטים שמגדירים את הדרישות מראש.

המודל מממש

מייצר קוד ומתקן לפי כישלונות

המפתח מגדיר

כותב טסטים שמבטאים את הדרישות

יחד תוצאה אמינה ומדויקת יותר

קוד שעובר טסטים ידועים וגם בדיקות נסתרות.

